

EECE4632 Hardware-Software Codesign for FPGA Based Systems

Final Project Report

David Noble & Clara Small

April 24, 2026

Project Description

Do-It-Yourself Guitar Pedals are an extremely popular hobby for electrically inclined guitar players, with numerous forums attracting thousands of weekly users. Typically hobbyists tend to mimic professional designs that you would see on consumer pedals, designing them using DSPs, or having designing the pedal completely in hardware. These designs have one major drawback: to add any more effects to a single pedal, you have to increase the amount of space needed. To counteract this we want to design a pedal using an FPGA that could act as an entire pedal board, mimicking multiple different effects and getting the same sounds without needing to have multiple pedals.

For our proof of concept design we are going to implement 4 different effects: Soft and Hard Clipping Distortion, Echo, Chorus, and Tremolo. We want to show that we are able to get the total delay through the pedals to less than 50ms regardless of the string of effects being done, which is fast enough that the player would not notice the delay in sound while playing. It will also demonstrate a significant speed up when compared to doing the same effects in pure software, seen 20 times improvement in delay. For both the testing on the hardware and in software we take an input wav file, converted to raw data. We then add our effects to this bitstream, then repackage it as a wav file to listen to the results. Our initial plan was to be able to have the output play directly off the FPGA, but due to time constraints we had to drop the part of the project.

Sound Effects

Hard Clipping Distortion: Works by applying a gain to boost the signal, then hard-cutting any values that exceed a set threshold. This produces a harsh, aggressive distortion characteristic of classic fuzz pedals.

Soft Clipping Distortion: Similar to hard clipping, but instead of abruptly cutting the signal, it uses a tanh function to smoothly compress values as they approach the threshold. This produces a warmer, less harsh distortion that more closely resembles tube amplifier overdrive.

Echo: Creates a delay effect by maintaining a circular buffer offset from the original signal by a set number of samples. The delayed signal is mixed back with the original, and a feedback parameter controls how many times the echo repeats before fading out.

Tremolo: Multiplies the audio signal by a low-frequency oscillator (LFO) which modulates the volume of the signal. The rate parameter controls how fast the volume oscillates, and the depth parameter controls how pronounced the effect is.

Chorus: Combines the original signal with a slightly delayed and pitch-modulated copy of itself to create a shimmering effect that makes a single instrument sound like multiple. Like the echo effect, it uses a circular buffer, but the delay time itself is continuously modulated by an LFO. Linear interpolation is applied between buffer samples to prevent audible clicking artifacts from the modulation.

Vitis Implementation

We started with our baseline Python code that we wrote for the very first project deadline, which implemented all 4 of our effects and tested them with a simple wav file. We then translated these effects from Python code into the C++ code for the hardware. This started with a simple code to test the audio input and output, which was inspired by the AXI-Stream example code we had for homework in class. We then built a do while loop inside the program, which computes the loop for each sample in the input audio_sample.

Then, we used the switches on the board as inputs, bundling them as a `s_axilite ap_uint<8>` and extracting the state of each switch as a `bool`. All of the switches corresponded to a different effect. We checked if each of them was on, and performed the effect on the sample if yes. This made it possible to stack our effects, as the sample was changed as the loop went on.

Hard clipping distortion, soft clipping distortion, and tremolo are done in `pedal_c.cpp` which are simple functions that return the altered audio sample. These functions were very similar to the Python implementation, just with different syntax. The echo and chorus effects had to be done in the top file because they required information from the previous samples, which we stored globally in buffers. On each loop, more data is saved in the buffers and the `sample_count` is incremented every time until it reaches the sample rate, where it resets to 0.

Testing and Results

The original recording and recordings after implementing the guitar effects can be found here: https://drive.google.com/drive/folders/1Ga9601ZqkzmuVqzdtSRaINajPa02avUf?usp=drive_link

Effects	Software time (s)	Hardware time (s)	Speed up
Hard Clip	0.754	0.0383	19.6 x Faster

Soft Clipped	0.616	0.0386	16 x Faster
Echo	0.362	0.0381	9.5 x Faster
Tremolo	0.602	0.0381	15.8 x Faster
Chorus	1.475	0.0392	37.6 x Faster
Hard + Echo	0.387	0.0379	10.2 x Faster
Soft + Chorus	1.440	0.0379	38 x Faster
Echo + Chorus	1.489	0.0379	39.3 x Faster
All	3.800	0.0380	100 x Faster
All - Tremolo	3.206	0.0383	83.7 x Faster

Figure 1: Table comparing hardware and software time

Comparing our results from software to hardware, we see significant improvements in speed, an average improvement of around 20x over the software implementation. If this were to be used as an actual guitar pedal, the delays we see in hardware are small enough that the user would likely not notice any latency, while they almost certainly would if the software implementation were used.

One interesting result is that regardless of the effect, the execution time remains nearly constant for the hardware implementation. This is likely because of how our pipeline was designed and optimized. Every sample passes through each stage whether the effect is active or not, so the pipeline depth is the same as if all effects were enabled. We did not implement a shortcut for the data to bypass unused stages. Doing so might decrease the time for individual effects, but would likely increase the hardware resource cost.

Hardware Exploration

Our code originally used the `cmath` library with many floating point variables, as we were focused on getting code we could start from. This had a huge area, so we updated our code to use `hls_math` instead of `cmath` and fixed<32,16> variables instead of floating points. This massively decreased our area. There were few variables we had to cast as floating points, because the math functions like `tanh` and `sinf` required this.

All of our code used loop flatten off and HLS pipeline II=1 because our effects process one audio sample per clock cycle in a streaming fashion, and pipeline II=1 ensures this throughput is maintained. Loop flattening was disabled because our loop structures use AXI Stream reads with TLAST termination, and flattening could introduce stalls or interfere with the

stream interface behavior.

One thing of note is the pragmas used for our buffer arrays: bind storage and dependence. These were added to resolve iteration interval warnings we were getting during HLS synthesis. Bind storage defines how many read and write ports are available per cycle. For the echo buffer, this is set to one read and one write, as it only needs to write the current sample to a new position and read from an older one. The chorus buffer requires two read ports because of the interpolation between two delay taps. The dependence false pragma tells Vitis that reads and writes on each buffer are always well separated in address space, which it would not have been able to determine on its own. Without this hint, HLS would conservatively assume a dependency and stall the pipeline, breaking our II=1 target.

Then, we started playing around with improving our math. We decreased the number of divisions in our code by making a division constant before calling it within a loop and using if statements for the modulus operator. We tried using a cubic function for soft clipping distortion instead of tanh, but the results were exactly the same for both so we went back to using tanh. The last thing we tried was using the fixed<32,16>.toint() instead of just casting these variables to ints. This one had a slightly bigger area but we ultimately decided on this implementation to get rid of the quantization in the tremolo effect.

A description of each change made can be seen below:

1. Using cmath and floating points, loop flatten off, pipeline II=1
Area: 110,819 blocks
2. Using hls_math and fixed<32,16>, loop flatten off, pipeline II=1
Area: 68974 blocks
3. Using hls_math and fixed<32,16>, loop flatten off, pipeline II=1, less division in soft clipping
Area: 48694 blocks
4. Using hls_math and fixed<32,16>, loop flatten off, pipeline II=1, less division in soft clipping, using cubic for soft clipped instead
Area: 48694 blocks
5. Using hls_math and fixed<32,16>, loop flatten off, pipeline II=1, less division in soft clipping, to_int()
Area: 65166 blocks

Hardware Design

To start our hardware design, we wanted to begin from a working prototype and build off it. This would minimize the time spent creating a basic overlay capable of sending and receiving data over AXI-Stream, letting us focus instead on the pedal-specific hardware. To do this, we took the AXI-Stream overlay we had used in class and copied it as a base. With the overlay made we began to explore how we could interact with different aspects of the board, such as LEDs, switches, and the audio codec, with the goal of being able to control what effects are being doing on the audio with the switches, and then have the output be sent to the audio codec to be listened

to in real time. In the end, partially because of time constraints and partially because of how long it took to test each design, we were only able to get the switches working and were not able access the audio codec.

The first hurdle we had to overcome was to figure out how to communicate with external components and pins on the board, which turned out to be extremely simple. For the switches we had to add an AXI GPIO IP, and then connect the GPIO pin to an external connection named “Switches”. All the other pins can be wired automatically by Vivado. We then had to create a constraints file and connect them to the switches pinout found in the AUP-ZU3 Reference Manual [1]. In the constraints file, we learned that when defining the pin connections you also have to define the I/O standard for the pins, which can be found in the reference manual. In our case this was LVCMOS12 (1.2 V). We also had to set the drive strength to 4 mA, which is required by Vivado's DRC for pins on this HP bank. One last thing to note is that the port name in the XDC file is switches_tri_io rather than switches. This suffix is generated automatically by the AXI GPIO IP and must be matched exactly.

This was our final design for the project, yet we still tried to get the audio codec working after. The following steps may not be the correct way because it didn't end up working, but they are the steps we followed.

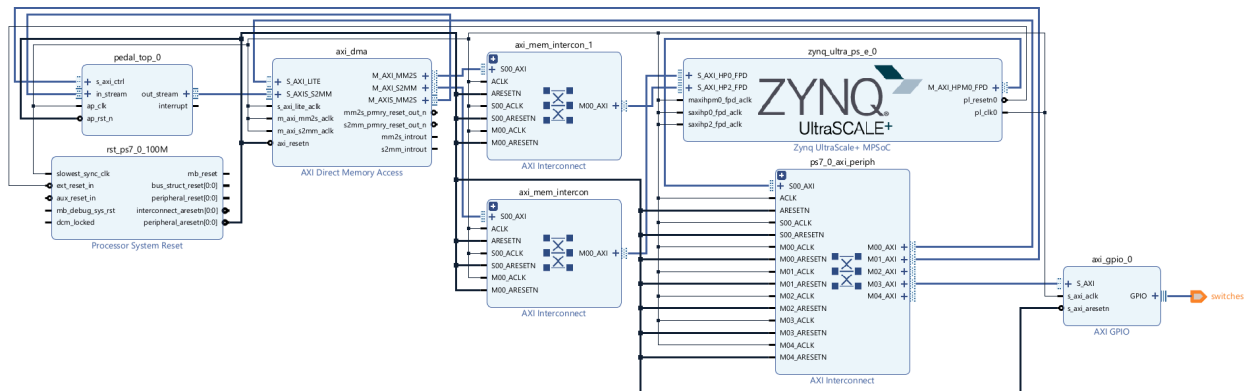


Figure 3: Final Hardware Design

To start, the data going into the audio codec has to be sent over I2S, which takes an input data stream and serializes it into a format the codec can convert into audio. This can be added using the I2S Transmitter IP, with all of its output pins set to external so they can connect to the audio codec. To drive the codec, we need a master clock that matches a multiple of the audio sample rate, which in our case was 11.2896 MHz (256 times the 44.1 kHz sample rate). This clock can be generated using a Clocking Wizard IP fed by the main PL clock. We then connect this output to the aud_mclk pin of the I2S Transmitter. We also need to add a second Processor System Reset block running in this new clock domain, and connect its output to the audio reset input on the I2S Transmitter. Note that in our project we did not do this, which we now believe to be a mistake. The remaining clock inputs on the I2S Transmitter, including the AXI-Stream

clock and the AXI-Lite control clock, should be driven by the same clock as the rest of the design.

With the I2S transmitter (maybe) working we then want to connect everything to the actual audio codec. We can do this exactly the same way we did it for the switches: by creating external ports, then defining what they are and where they connect in the constraints file. The pin assignments for the audio codec are also in Appendix A of the reference manual [1]. The four main signals we need to connect are sdata (the serial audio data going out to the codec), lrcclk (the left/right channel clock), sclk (the serial bit clock), and the master clock output from the Clocking Wizard, which maps to the codec's MCLK pin. All four of these are on the LVCMOS18 (1.8 V) bank. We also need to drive the codec's reset pin, AIC_nRST. We drove this high using a Constant IP in the block design, which in theory should let the codec turn on since the n in AIC_nRST indicates an active-low reset.

There are a couple of possible issues that could be the reason we never got the codec working. The first is that we don't actually know if we set up the I2S transmitter correctly. When we started working on this, we did not find any example projects or much documentation on how to set up this IP. In hindsight it would have been worthwhile to spend longer looking for these resources and reading through the IP's documentation directly. We mainly relied on AI for learning how the tool works, and it is possible that this was not the right approach. After finishing the project we found a Reddit thread discussing how to set up the audio codec [2]. Their approach seemed very different from ours, so with more time it would have been worth exploring.

Unfortunately, it became impossible to test different ways of getting the codec to function while also making progress on the rest of the design, because whenever the codec failed it would kill the AXI-Stream, causing it to lock up and stop sending data. This meant we could not test both at the same time, so any further exploration of the audio codec would have had to happen separately. With bitstream generation taking around 30 minutes per iteration, time constraints got the better of us and we had to drop it.

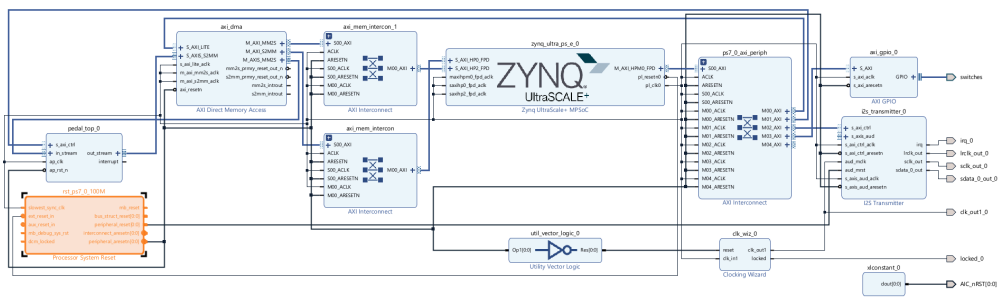


Figure 4: Design for Audio Codec

Lessons Learned/Future Considerations

Throughout this project, we learned that creating multiple bitstreams for trial and error can take upwards of 30 minutes. Other issues we ran into when experimenting include the AXI

pipelining freezing when the audio codec didn't work. This made it difficult to test both the audio codec connection and the guitar pedal effects, and we didn't have enough time to do them separately. Next time, we would definitely do these separately, or create a better method of testing the pedal effects in Vitis so we didn't have to rely on listening to the output sound.

We also learned a lot about how to code for hardware, improving our skills with hardware exploration and the best designs for our board. The fixed point variables really helped our design in terms of area, and so did the hls_math library (which we previously didn't know existed). We also learned how to connect the board switches to our design in Vivado and Vitis to use them as inputs.

The next steps in our project would be to get the audio codec working, and to fix the tremolo quantization.

How we used AI

We used AI to write a testbench to verify soft clipping, hard clipping, and echo in Vitis. This used a simulated sine wave and was right before the project 2 deadline. We wanted a quick check before testing these effects with the actual wav file in hardware.

We also used AI to give some suggestions for hardware exploration. For example, it gave us some ideas for improving our math functions, like eliminating divisions and the modulus operator. It also gave the suggestion of using the cubic function instead of the tanh function.

We used AI to help us debug, mostly with the Vivado project. It was used to help try and read Vivado error messages, and give ideas on how we could set up our codec. It also helped to tell us which IP to use to access the switches. In the end, it would likely have been more beneficial to mimic other projects directly than to have AI try to coach us on how to set up the codec.

Citations

[1] Real Digital, "AUP-ZU3 Reference Manual," *realdigital.org*.
<https://www.realdigital.org/downloads/3cd0cbae4d00eaf94270e5792c746d4c.pdf> (accessed Apr. 24, 2026)

Further Reading

[2] Reddit thread about audio connections in Vivado:
https://www.reddit.com/r/FPGA/comments/1i15pr9/help_with_i2s_ip_core_setup_in_vivado_no_processor/?solution=1a57d52f0415927a1a57d52f0415927a&js_challenge=1&token=bbbe4bf1c9a2b5160829c4be34da5861f4a0301af8b00f59661d9425b91284b5